Kapitel 1

Warum Softwaretechnik?

Agenda

Abschnitt 1.1

Software & Softwaresysteme

Abschnitt 1.2

Software-Desaster

Abschnitt 1.3

Softwareentwicklung & Software Engineering

Abschnitt 1.4

Softwarequalität & Softwarekosten

Klausurrelevante Lernziele

- Phasen der Softwareentwicklung aufzählen und mindestens drei näher erläutern können
- Wichtige Stakeholder bei der Softwareentwicklung aufzählen können
- Ziele des Software Engineering erläutern können
- Bedeutung von Standard- und Individualsoftware erläutern und einige Vor- und Nachteile nennen können
- Zusammenhang der Phasen von Verursachung und Entdeckung von Softwarefehlern kennen
- Gründe für eine möglichst frühe Qualitätssicherung nennen können
- Zentrale Aspekte von Softwarequalität erläutern können
- Mindestens vier Hauptmerkmale nach ISO/IEC 25010 aufzählen und erläutern können

Abschnitt 1.1

Software & Softwaresysteme

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Software

- »Hardware« für Eisenwaren seit 16. Jahrhundert
- »Software« ist Kunstwort von John W. Tukey (1957)

Software – Computer programs, procedures and possibly associated documentation and data pertaining to the operation of a computer system.

(ISO/IEC/IEEE 24765:2017)

- Software
  - … ist immateriell
  - … wird nicht gefertigt, sondern entwickelt
  - … ist beliebig reproduzierbar: Original und Kopie sind exakt identisch
  - … ist leichter änderbar als ein technisches Produkt
  - … verschleißt nicht: Fehler sind bereits enthalten oder werden durch Änderung hinzugefügt
  - … altert (»erodiert«) trotzdem, weil Umgebung & Anforderungen der Nutzenden sich ändern

Erstes Computerprogramm (1843)

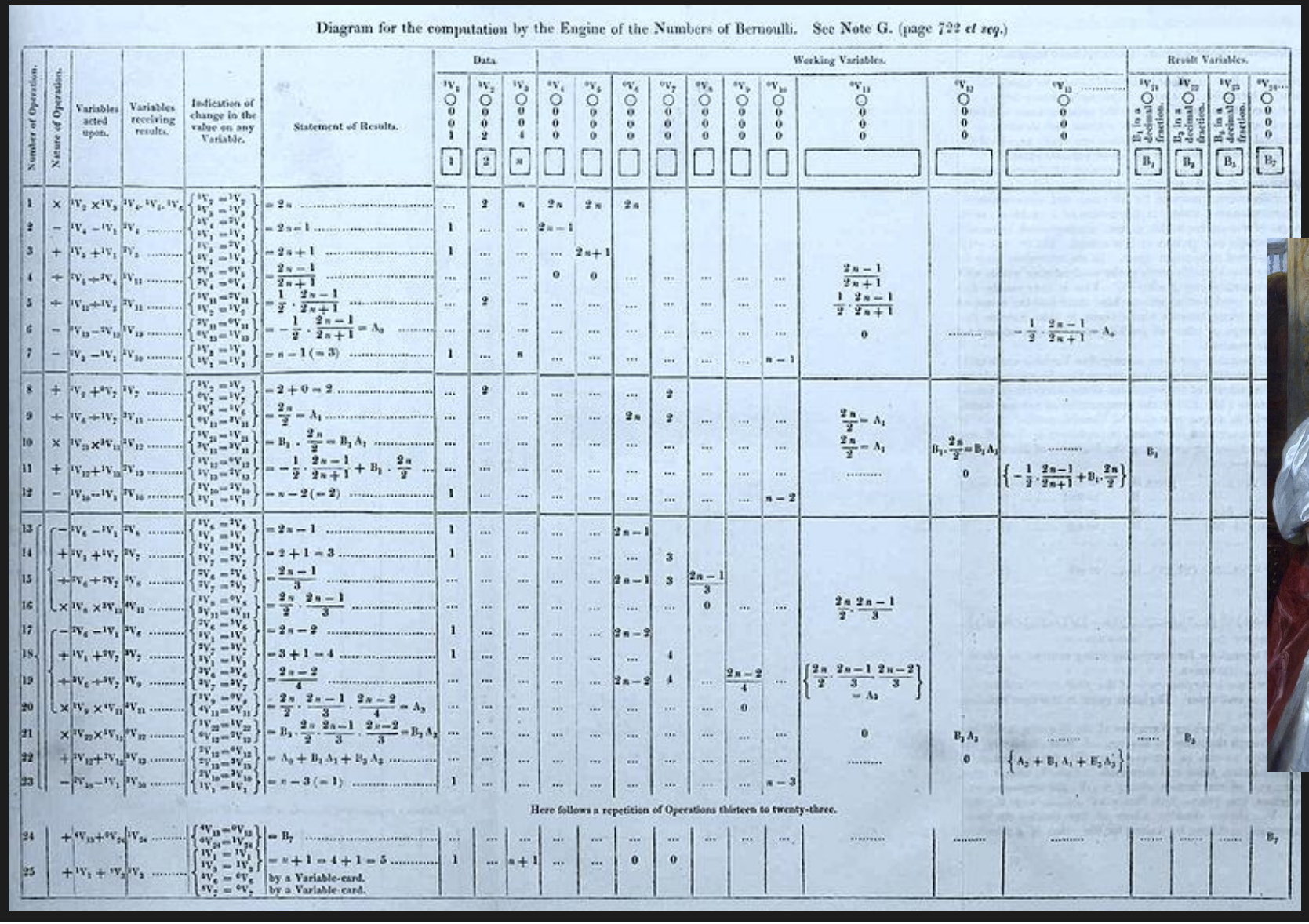


Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 *et seq.*)

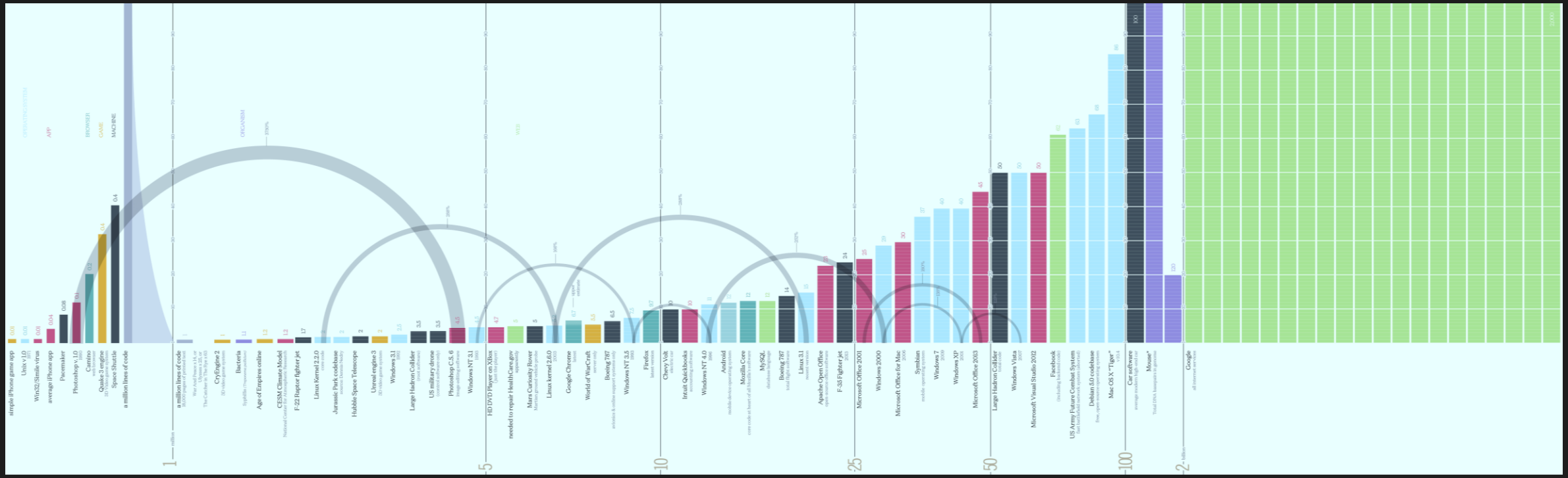
Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data. $^1V_1$ = 1	Data. $^1V_2$ = 2	Data. $^1V_3$ = n	Working Variables. $^2V_4$	$^2V_5$	$^2V_6$	$^2V_7$	$^2V_8$	$^2V_9$	$^2V_{10}$	$^2V_{11}$	$^2V_{12}$	$^2V_{13}$	Result Variables. $^1V_{21}$ B_1 in a decimal fraction. B_1	$^1V_{22}$ B_2 in a decimal fraction. B_2	$^1V_{23}$ B_3 in a decimal fraction. B_3	$^2V_{24}$ ... B_7
1	×	$^1V_2 \times {}^1V_3$	$^2V_4, {}^1V_5, {}^1V_6$	$\begin{cases} {}^1V_2 = {}^1V_2 \\ {}^1V_3 = {}^1V_3 \end{cases}$	$= 2n$	...	2	n	$2n$	$2n$	$2n$											
2	−	$^1V_4 - {}^1V_1$	$^2V_4$	$\begin{cases} {}^1V_4 = {}^2V_4 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n - 1$	1	...	...	$2n-1$													
3	+	$^1V_2 + {}^1V_1$	$^2V_5$	$\begin{cases} {}^1V_2 = {}^2V_5 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n + 1$	1	...	...	...	$2n+1$												
4	÷	$^2V_5 \div {}^2V_4$	$^2V_{11}$	$\begin{cases} {}^2V_5 = {}^6V_5 \\ {}^2V_4 = {}^6V_4 \end{cases}$	$= \frac{2n-1}{2n+1}$	...	...	...	0	0	...	...	...	...	...	$\frac{2n-1}{2n+1}$						
5	÷	$^1V_{12} \div {}^1V_2$	$^2V_{31}$	$\begin{cases} {}^1V_{12} = {}^2V_{31} \\ {}^1V_2 = {}^1V_2 \end{cases}$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1}$	...	2	...	...	...	...	...	...	...	...	$\frac{1}{2} \cdot \frac{2n-1}{2n+1}$						
6	−	$^2V_{13} - {}^2V_{12}$	$^1V_{13}$	$\begin{cases} {}^2V_{13} = {}^6V_{31} \\ {}^6V_{13} = {}^1V_{13} \end{cases}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$	...	...	...	...	...	...	...	...	...	...	0	.....	$-\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$				
7	−	$^1V_3 - {}^1V_1$	$^2V_{10}$	$\begin{cases} {}^1V_3 = {}^1V_3 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= n - 1\ (= 3)$	1	...	n	...	...	...	...	...	...	$n-1$							
8	+	$^1V_2 + {}^2V_7$	$^2V_7$	$\begin{cases} {}^1V_2 = {}^1V_2 \\ {}^2V_7 = {}^1V_7 \end{cases}$	$= 2 + 0 = 2$	...	2	...	...	...	...	2										
9	÷	$^1V_6 \div {}^1V_7$	$^2V_{13}$	$\begin{cases} {}^1V_6 = {}^1V_6 \\ {}^2V_{13} = {}^2V_{31} \end{cases}$	$= \frac{2n}{2} = A_1$	...	...	...	...	...	$2n$	2	...	...	...	$\frac{2n}{2} = A_1$						
10	×	$^1V_{22} \times {}^2V_{12}$	$^2V_{12}$	$\begin{cases} {}^1V_{22} = {}^1V_{21} \\ {}^2V_{12} = {}^2V_{31} \end{cases}$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...	...	...	...	...	...	...	...	...	$\frac{2n}{2} = A_1$	$B_1 \cdot \frac{2n}{2} = B_1 A_1$	.....	B_1			
11	+	$^1V_{12} + {}^1V_{13}$	$^2V_{13}$	$\begin{cases} {}^1V_{12} = {}^6V_{12} \\ {}^1V_{13} = {}^2V_{13} \end{cases}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}$ ...	...	...	...	...	...	...	...	...	...	...	.....	0	$\left\{ -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2} \right\}$				
12	−	$^1V_{10} - {}^1V_1$	$^2V_{10}$	$\begin{cases} {}^1V_{10} = {}^2V_{10} \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= n - 2\ (= 2)$	1	...	...	...	...	...	...	...	...	$n-2$							
13	−	$^1V_6 - {}^1V_1$	$^2V_6$	$\begin{cases} {}^1V_6 = {}^2V_6 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n - 1$	1	...	...	...	...	$2n-1$											
14	+	$^2V_3 + {}^1V_7$	$^2V_7$	$\begin{cases} {}^1V_3 = {}^1V_3 \\ {}^1V_7 = {}^2V_7 \end{cases}$	$= 2 + 1 = 3$	1	...	...	...	...	...	3										
15	÷	$^2V_6 \div {}^2V_7$	$^2V_6$	$\begin{cases} {}^2V_6 = {}^2V_6 \\ {}^2V_7 = {}^2V_7 \end{cases}$	$= \frac{2n-1}{3}$	...	...	...	...	...	$2n-1$	3	$\frac{2n-1}{3}$									
16	×	$^1V_8 \times {}^2V_{12}$	$^2V_{11}$	$\begin{cases} {}^1V_8 = {}^6V_8 \\ {}^2V_{11} = {}^1V_{11} \end{cases}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3}$	...	...	...	...	...	...	...	0	...	...	$\frac{2n}{2} \cdot \frac{2n-1}{3}$						
17	−	$^2V_6 - {}^1V_1$	$^2V_6$	$\begin{cases} {}^2V_6 = {}^3V_6 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n - 2$	1	...	...	...	...	$2n-2$											
18	+	$^1V_1 + {}^2V_7$	$^2V_7$	$\begin{cases} {}^2V_7 = {}^3V_7 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 3 + 1 = 4$	1	...	...	...	...	...	4										
19	÷	$^3V_6 \div {}^3V_7$	$^1V_9$	$\begin{cases} {}^3V_6 = {}^3V_6 \\ {}^2V_7 = {}^2V_7 \end{cases}$	$= \frac{2n-2}{4}$	...	...	...	...	...	$2n-2$	4	...	$\frac{2n-2}{4}$	...	$\left\{ \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{3} = A_2 \right\}$							
20	×	$^1V_9 \times {}^1V_{11}$	$^2V_{11}$	$\begin{cases} {}^1V_9 = {}^6V_9 \\ {}^4V_{11} = {}^6V_{11} \end{cases}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = A_2$	...	...	...	...	...	...	...	...	0								
21	×	$^2V_{22} \times {}^2V_{12}$	$^2V_{32}$	$\begin{cases} {}^1V_{22} = {}^1V_{22} \\ {}^6V_{12} = {}^2V_{12} \end{cases}$	$= B_2 \cdot \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{3} = B_2 A_2$	...	...	...	...	...	...	...	...	...	...	0	$B_2 A_2$	.....	.....	B_2		
22	+	$^2V_{12} + {}^2V_{13}$	$^2V_{13}$	$\begin{cases} {}^2V_{12} = {}^6V_{12} \\ {}^2V_{13} = {}^2V_{13} \end{cases}$	$= A_0 + B_1 A_1 + B_2 A_2$	...	...	...	...	...	...	...	...	...	...	.....	0	$\left\{ A_2 + B_1 A_1 + B_2 A_2 \right\}$				
23	−	$^2V_{10} - {}^1V_1$	$^2V_{10}$	$\begin{cases} {}^2V_{10} = {}^2V_{10} \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= n - 3\ (= 1)$	1	...	...	...	...	...	...	...	...	$n-3$							
Here follows a repetition of Operations thirteen to twenty-three.																						
24	+	$^6V_{13} + {}^6V_{24}$	$^1V_{24}$	$\begin{cases} {}^6V_{13} = {}^6V_{13} \\ {}^6V_{24} = {}^1V_{24} \end{cases}$	$= B_7$	...	...	...	...	...	...	...	...	...	...	.....	.....	.....	.....	.....	.....	B_7
25	+	$^1V_1 + {}^1V_3$	$^1V_3$	$\begin{cases} {}^1V_1 = {}^1V_1 \\ {}^1V_3 = {}^1V_3 \\ {}^4V_6 = {}^6V_6 \\ {}^6V_7 = {}^6V_7 \end{cases}$	$= n + 1 = 4 + 1 = 5$ ..... by a Variable-card. by a Variable-card.	1	...	$n+1$	...	...	0	0										

Ada Lovelace
(1815-1852)

Kleine Geschichte der Softwareentwicklung

- 1960er
  - Dijkstra 1968: »Go To Statement Considered Harmful«
  - Structured Programming
  - »Software Crisis«: einige Großprojekte scheitern
- 1970er
  - Royce 1970: Software Life Cycle
  - Rochkind 1972: Source Code Control System (SCCS)
  - Nassi-Shneiderman-Diagramme (Struktogramm); abstrakter Datentyp; Datenkapselung; Information Hiding
  - Ross 1977: Structured Analysis Design Technique (SADT)
- 1980er
  - Konzept der integrierten Entwicklungsumgebung (Integrated Development Environment, IDE): Editor, Compiler, Linker & Debugger
  - Objektorientierte Programmierung
- 1990er
  - Objektorientierte Analyse- und Entwurfsmethoden, insbesondere UML

Komplexität von Software: Lines of Code (LOC)

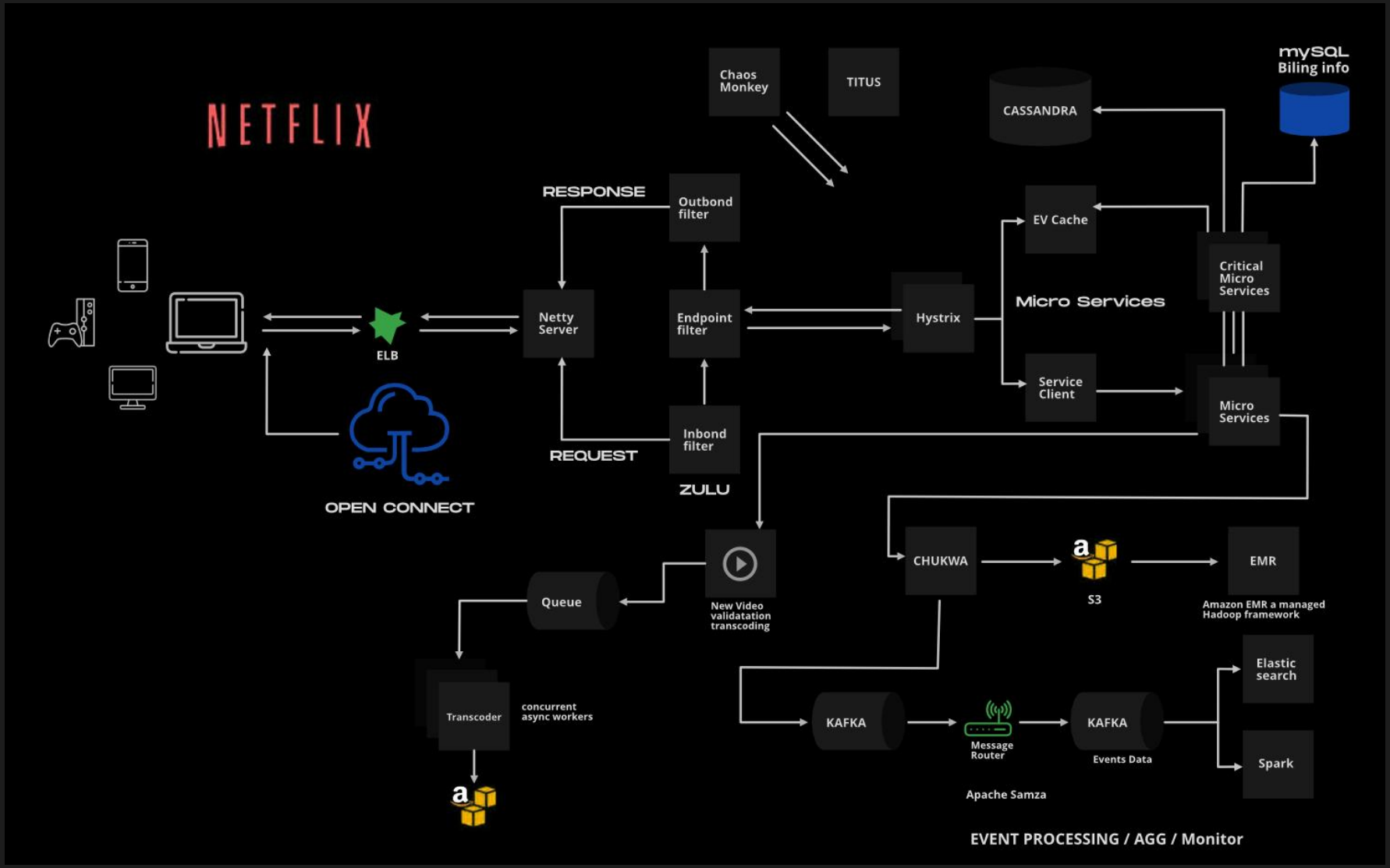


Quelle: www.informationisbeautiful.net (2015)

Moderne Softwaresysteme

- Steigende technische Komplexität
  - Millionen Codezeilen, Zehntausende Klassen, Hunderte Komponenten
  - Multiparadigmatische Programmierung & mehrere Programmiersprachen
  - Verschiedene Technologien, Konzepte, Muster
  - Verschiedene Datenbank-Ansätze: relational, NoSQL
  - Verschiedene Endgeräte: Fat Client, Web Client, Mobile
  - Leistungsfähige Frameworks, weniger »Boilerplate-Code«
  - Komplexe Architekturen: Client / Server, Schichten, verteilte Services, SOA, Microservices etc.
  - Komplexes Hosting: Cloud, on-premise, hybrid
  - Komplexe Ergonomie: responsive Design
  - Komplexe Authentifizierung & Autorisierung, Logging, Performance-Optimierung, Suche, Reporting etc.
- Steigende Anforderungen an Entwicklung & Betrieb
  - Verteilte Entwicklung, viele Entwickelnde in internationalen Teams
  - Agile Softwareentwicklung, Continuous Integration (CI), Continuous Delivery (CD), DevOps
  - Betrieb auf Basis von Container-Technologien

Beispiel: Softwarearchitektur von Netflix



Quelle: https://www.geeksforgeeks.org/system-design-netflix-a-complete-architecture/ (2024)

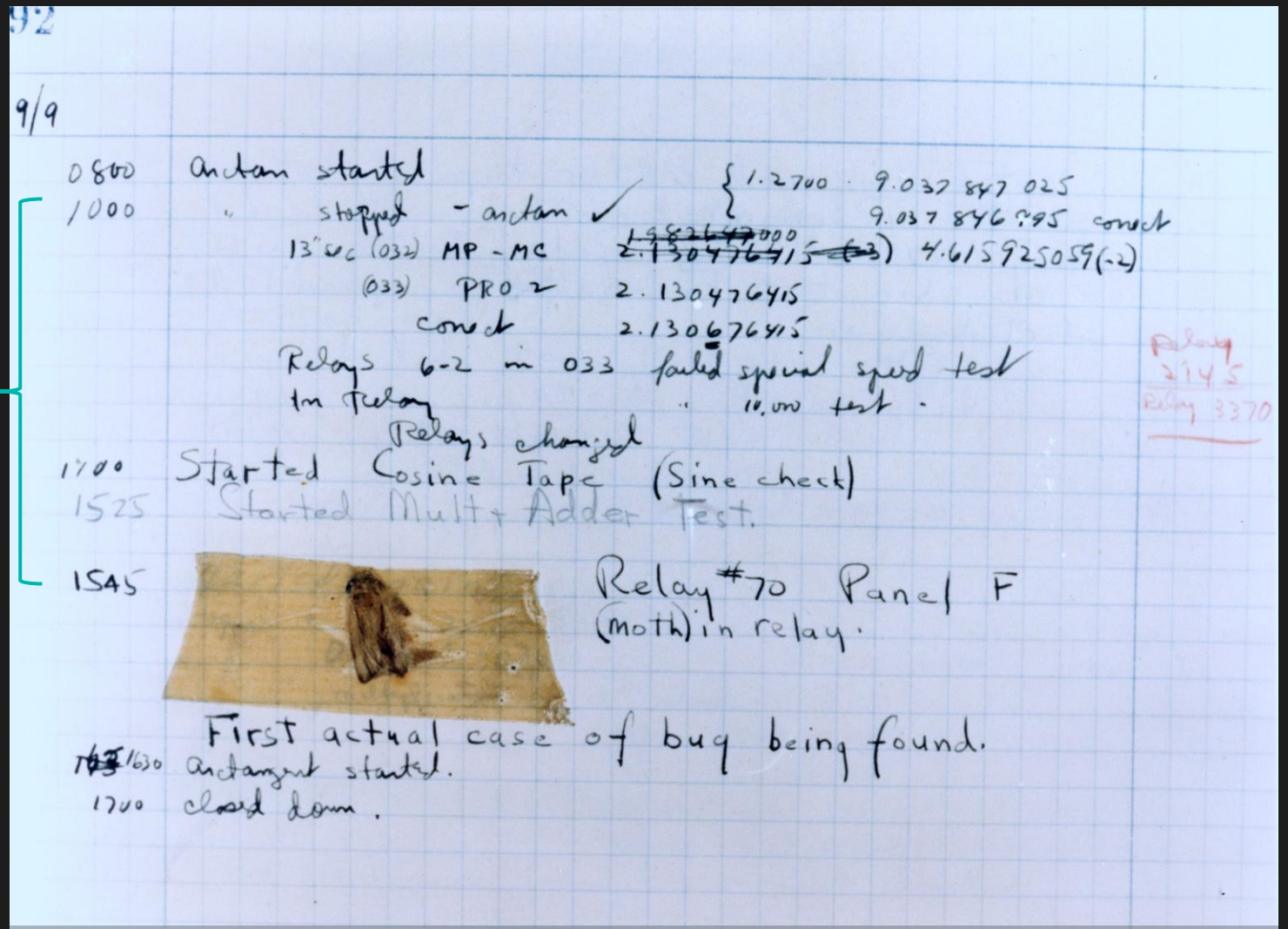
Abschnitt 1.2

Software-Desaster

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Der erste Bug (1947)

Knapp 6 Stunden »Debugging«



Logbuch Mark II Aiken Relay Calculator

Grace Hopper
(1906 – 1992)

Software-Desaster (1)

- 1962: Mariner 1-Sonde zur Erkundung der Venus
  - Selbstzerstörung nach 293 Sekunden: vergessener Überstrich aus handschriftlicher Notiz
- 1965: Gemini V, bemannter Weltraumflug der NASA
  - Landplatz um 145 km verfehlt: Rotation Erde um Sonne wurde nicht berücksichtigt
- 1979: Amerikanisches Abwehrsystem meldet Sowjet-Angriff
  - NORAD meldet 2020 russische Interkontinentalraketen: Defekt in Software, zufällige Bits
- 1983: Russisches Abwehrsystem meldet US-Angriff
  - 0:30 Uhr: System meldet 5 US-Interkontinentalraketen; Software hatte Sonnenreflexionen auf Wolken über Air Force Base als Raketen interpretiert; Oberstleutnant Petrow entscheidet im Geheimbunker »Serpuchow 15« aufgrund eines Bauchgefühls: kein Gegenschlag!
- 1996: Jungfernflug der Ariane 5
  - Selbstzerstörung nach 37 Sekunden: Navigationssystem von Ariane 4 übernommen, falsche Umwandlung der Geschwindigkeit (64-Bit-Gleitkommazahl in ganzzahligen 16-Bit-Wert)
- 2005: Kabel zur kurz für Airbus 380
  - Hamburg: Elektronik, Version 4, Fortran/Unix; Toulouse: Rumpf, Version 5, C++/Windows
  - CAD-Software: Metadaten bei Konvertierung gingen verloren

Software-Desaster (2)

TU Dresden

Uni verschickt Exmatrikulations-Mails an alle Studenten

Eine fehlerhafte E-Mail aus dem Rechenzentrum hat 37.000 Studenten der TU Dresden aufgeschreckt. Jeder einzelne hatte eine Nachricht über seine Exmatrikulation erhalten. Auch Uni-Mitarbeiter befürchteten ihre Entlassung.

Von **Fabienne Kinzelmann**
11.11.2013, 12.19 Uhr

Quelle: Der Spiegel (2013)

Software-Desaster (3)

Ort Bonn
Datum 16.12.2021

Quelle: ©olm26250 /iStock Getty Images Plus / Getty Images

Die Schwachstelle namens „Log4Shell" in der weit verbreiteten Java-Bibliothek Log4j führt nach Einschätzung des Bundesamts für Sicherheit in der Informationstechnik (BSI) weiterhin zu einer kritischen IT-Sicherheitslage. Das BSI stellt aktuelle Informationen unter › www.bsi.bund.de/dok/log4j zur Verfügung.

Nach wie vor besteht keine abschließende Klarheit darüber, welche IT-Produkte durch „Log4Shell" verwundbar sind. Einen Überblick über den Verwundbarkeitsstatus zahlreicher IT-Produkte pflegt die niederländische Partnerbehörde des BSI, zu der auch das BSI selbst beiträgt.

Die Schwachstelle wird aktuell mit unterschiedlichen Angriffsformen weltweit ausgenutzt. Neben Angriffen mit Krypto-Minern (dadurch werden die betroffenen Systeme zur Errechnung von Krypto-Währungen missbraucht) oder Bot-Netzen (die betroffenen Systeme werden in Bot-Netze integriert, mit denen bspw. › DDoS-Angriffe durchgeführt werden), sind mittlerweile auch die ersten › Ransomware-Angriffe bekannt geworden. Bei Ransomware-Angriffen werden Computer oder ganze Netzwerke verschlüsselt und die Betroffenen um Lösegeld erpresst.

Aus Sicht des BSI ist mit einer breiten Ausnutzung der Schwachstelle und mit weiteren erfolgreichen Cyber-Angriffen zu rechnen. Diese können auch noch in einigen Wochen und Monaten folgen, wenn die genannte Schwachstelle jetzt für eine Erstinfektion genutzt wird.

Es ist daher weiterhin wichtig, die vom BSI empfohlenen IT-Sicherheitsmaßnahmen schnellstmöglich umzusetzen. Sofern Sicherheits-Updates für verwundbare IT-Produkte zur Verfügung stehen, sollten diese durch alle Anwenderinnen und Anwender eingespielt werden. Daher sind insbesondere Hersteller von IT-Produkten gefordert, ihre Produkte zu prüfen und sie gegebenenfalls durch Sicherheits-Updates abzusichern.

Datum 19.07.2024

Quelle: © xy / Fotolia.com

Weltweit kommt es derzeit zu IT-Ausfällen in zahlreichen Branchen. Auch in Deutschland gibt es betroffene Unternehmen, darunter Betreiber Kritischer Infrastrukturen.

Eine Ursache dafür ist offenbar ein fehlerhaftes Update einer IT-Security-Lösung des Herstellers Crowdstrike (Falcon). Diese wird von zahlreichen weiteren IT-Diensten genutzt, die in der Folge ausfallen. Nach Informationen des BSI hat der Hersteller der betroffenen Lösung einen Workaround kommuniziert, der von Betroffenen umgesetzt werden sollte. Das BSI erreichen Meldungen von Unternehmen, die den Workaround erfolgreich umsetzen konnten.

Darüber hinaus hat Microsoft kommuniziert, dass es in der Microsoft-Lösung Azure zu einem Konfigurationsfehler gekommen ist, der ebenfalls weltweit Auswirkungen zeigt. Ob und inwiefern die Vorfälle zusammenhängen, ist noch nicht abschließend geklärt.

Nach aktuellem Erkenntnisstand aus den Äußerungen der betroffenen Unternehmen gibt es keine Hinweise auf einen Cyberangriff. Die Vorfälle werden laufend weiter bewertet.

Das BSI hat seinen Zielgruppen entsprechende Informationen zukommen lassen und stellt zur Verfügung.

Quelle: BSI (2021), BSI (2024)

Abschnitt 1.3

Softwareentwicklung & Software Engineering

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Analogien für die Softwareentwicklung

- Seit über 70 Jahren: Softwareentwicklung orientiert sich oft am Hausbau
- Viele Konzepte und Begriffe wurden aus dem Bauwesen übernommen
- Bauwesen: sehr lange Tradition, sehr viel Erfahrung, fast immer klar abgegrenzte Phasen und Rollen (Auftraggebende, Architekturschaffende, Gewerke etc.)

Kunden-bedürfnisse und Rahmen-bedingungen verstehen

In Baupläne übersetzen

Haus bauen (vom Fundament zum Dach)

- Weitere Analogien: Handwerkskunst (Craftsmanship)? Garten pflegen? Gericht kochen? Drehbuch schreiben?

Quelle: (Martin 2025; Thomas, Hunt 2019; Freeman, Pryce 2009)

Softwareentwicklung: Prozess oder Projekt?

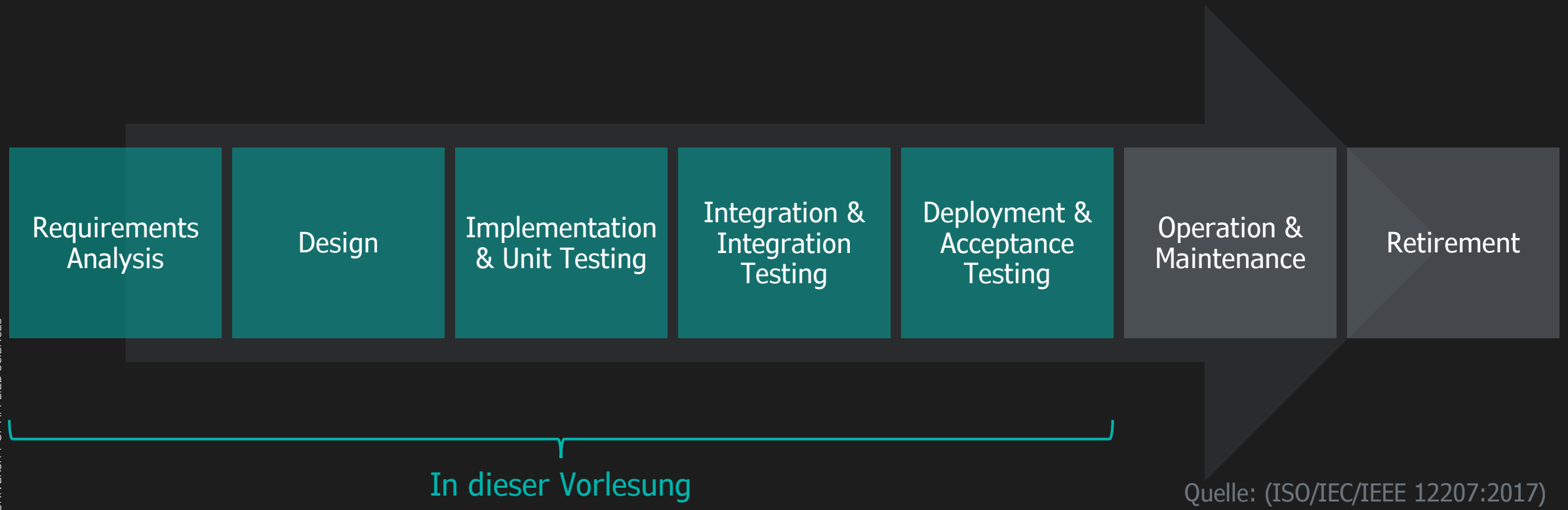
Software Development Process – Process by which user needs are translated into a software product. The process involves translating user needs into software requirements, transforming the software requirements into design, implementing the design in code, testing the code, and sometimes, installing and checking out the software for operational use. These activities can overlap or be performed iteratively.

(ISO/IEC/IEEE 24765:2017)

- Softwareentwicklung als Projekt?
  - Teilweise: Entwicklung kundenspezifischer Individualsoftware (»Make«)
  - Fast immer: Einführung einer Standardsoftware (»Buy«)
  - Achtung: Softwareentwicklung ist meistens kein Projekt, da Ziele nicht eindeutig sind und es wegen Wartung & Weiterentwicklung kein festes Enddatum gibt

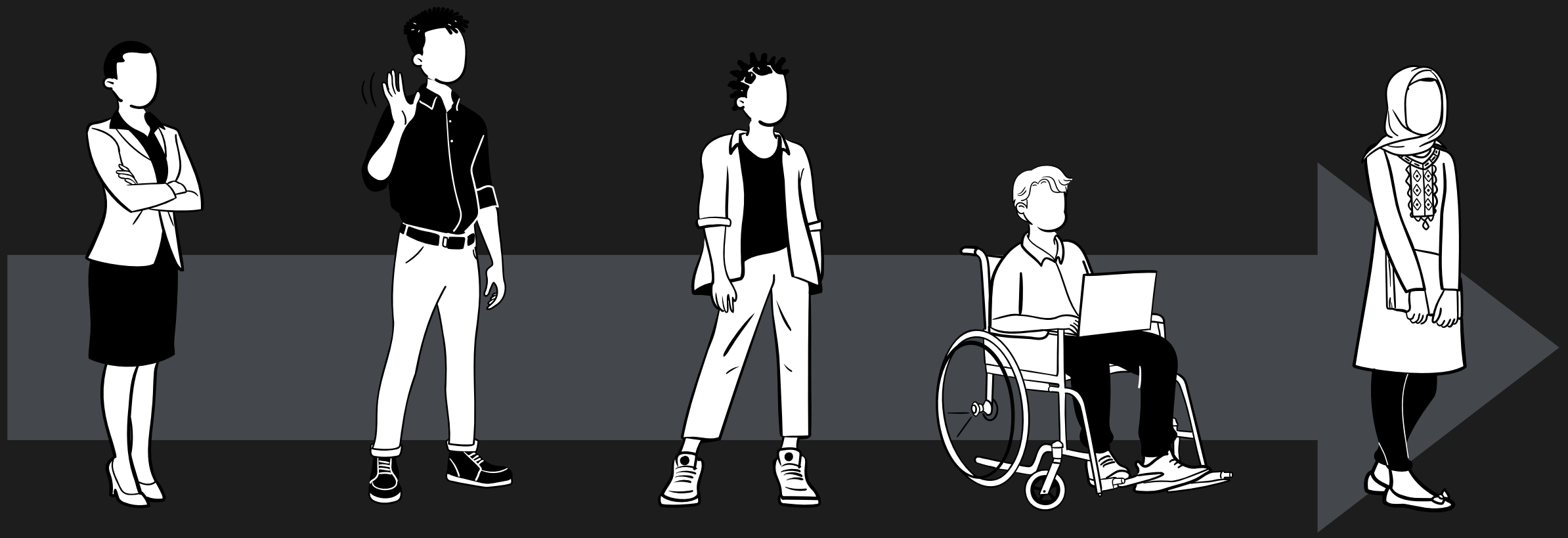
Phasen der Softwareentwicklung

- Auch: Software Development Life Cycle (SDLC)
- Üblicherweise in Phasen eingeteilt, je nach Vorgehensmodell [→ Kapitel 2]
- In Literatur nicht einheitlich, hier nach ISO-Norm
- Noch offen: alle Phasen hintereinander? Überlappungen? Wiederholungen?



Quelle: (ISO/IEC/IEEE 12207:2017)

Softwareentwicklung: Stakeholder



Anforderungs-erhebend

Architektur-schaffend

Entwickelnd

Testend

Betreibend

Außerdem

Auftraggebend

Supportend

Nutzend

Software Engineering (1)

- Ursprung: »Software Crisis« in 1960er
- »The whole trouble comes from the fact that there is so much tinkering with software. It is not made in a clean fabrication process, which it should be. What we need, is software engineering.« – F. L. Bauer, NATO-Tagung 1968, Garmisch-Partenkirchen
- Beobachtung
  - Bedeutung der Wahl von Programmiersprachen, Algorithmen & Datenstrukturen nimmt ab
  - Bedeutung von Entwurf & Beschreibung grundlegender Systemstrukturen nimmt zu
  - Weitergehende Kompetenzen nötig, um Komplexität von modernen Systemen zu beherrschen
- Betrachtung von Software als technisches Produkt
  - Wird von »Ingenieur:innen« im Team systematisch & wirtschaftlich entwickelt [→ Egoless Programming]
  - Ist gekennzeichnet durch überprüfbare Eigenschaften in Bezug auf Funktionalität, Zuverlässigkeit & Qualität

Software Engineering (2)

Software Engineering – (1) Systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software.

(2) Application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

(ISO/IEC/IEEE 24765:2017)

- … erschafft nichts, sondern verhindert Schäden
- … ist keine Technologie, sondern Verhaltenskodex für alle, die an Entwurf, Entwicklung & Betrieb von Software beteiligt sind
- … setzt effektiv Standards, Werkzeuge, Methoden & Vorgehensmodelle ein
- … sorgt für bedarfsgerechte Dokumentation
- … ist kein festes »Regelwerk«, sondern liefert kontextabhängige Empfehlungen

Software Engineering: Werkzeugeinsatz

- Computer-Aided Software Engineering (CASE)
- Werkzeuge zur Unterstützung von
  - Requirements Analysis
  - Design
  - Implementation
  - Unit Testing
  - Integration & Integration Testing
  - Konfigurationsmanagement
  - Software- und Testmetriken
  - Dokumentation
  - Projektmanagement

Upper CASE

Lower CASE

- Werkzeuge können Bestandteil einer integrierten Entwicklungsumgebung (IDE) oder eigenständig sein

Software Engineering: wichtige Vertreter

Jeff Sutherland
(* 1941)

Ken Schwaber
(* 1945)

Howard G. Cunningham
(* 1949)

Robert C. Martin
(* 1952)

Kent Beck
(* 1961)

Martin Fowler
(* 1963)

Prompt Engineering

»Prompt Engineering is the NEW Software Engineering«

»Vollautomatisierte Software-Entwicklung?« ist eine der fünf Grand Challenges 2025 der Gesellschaft für Informatik (GI e.V.)

Abschnitt 1.4

Softwarequalität & Softwarekosten

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Softwarequalität

Software Quality – (1) Capability of software product to satisfy stated and implied needs when used under specified conditions.

(2) Degree to which a software product satisfies stated and implied needs when used under specified conditions.

(ISO/IEC/IEEE 24765:2017)

- Unterschied von Software zu »normalen« technischen Produkten
  - Software ist immateriell: sie wird nicht gefertigt, sondern entwickelt
  - Software ist reproduzierbar: Original und Kopie sind exakt identisch
  - Software verschleißt nicht: Fehler sind bereits enthalten oder werden durch Änderung hinzugefügt (sie »erodiert« trotzdem, weil Umgebung & Anforderungen sich ändern)
- Qualität ist nur indirekt und für einzelne Eigenschaften messbar (etwa: Zeit- oder Ressourceneffizienz)
- Verschiedene Stakeholder haben unterschiedliche Qualitätsbedürfnisse: diese müssen als nicht-funktionale Anforderungen (NFA) explizit formuliert werden

ISO/IEC 25010:2023 – Characteristics (1)

- Funktionale Eignung (Functional Suitability)
  - Softwaresystem stellt Funktionen bereit, um (unter festgelegten Bedingungen) die angegebenen und vorausgesetzten Bedürfnisse der vorgesehenen Nutzenden zu erfüllen
- Leistungseffizienz (Performance Efficiency)
  - Softwaresystem führt die Funktionen innerhalb festgelegter Zeit- und Durchsatzparameter aus; Ressourcen werden (unter festgelegten Bedingungen) effizient genutzt
- Kompatibilität (Compatibility)
  - Softwaresystem kann Informationen mit anderen Produkten austauschen und erforderlichen Funktionen ausführen, während eine gemeinsame Umgebung bzw. gemeinsame Ressourcen geteilt werden
- Interaktionsfähigkeit (Interaction Capability)
  - Vorgesehene Nutzende können mit dem Softwaresystem über eine Bedienungsoberfläche interagieren und die beabsichtigten Aufgaben ausführen
- Zuverlässigkeit (Reliability)
  - Erforderliche Funktionen können (unter festgelegten Bedingungen) in einem bestimmten Zeitraum ohne Unterbrechung oder Ausfälle ausgeführt werden

Quelle: (ISO/IEC 25010:2023)

ISO/IEC 25010:2023 – Characteristics (2)

- Informationssicherheit & Datenschutz (Security)
  - Informationen und Daten werden so geschützt, dass autorisierte Personen bzw. Produkte einen ihrer Berechtigungsstufe angemessenen Datenzugriff besitzen, während Angriffe böswilliger Akteure abgewehrt werden
- Wartbarkeit (Maintainability)
  - Softwaresystem kann von dem vorgesehenen Wartungspersonen effektiv und effizient modifiziert werden
- Flexibilität (Flexibility)
  - Softwaresystem kann an geänderte Anforderungen, Nutzungskontexte oder Systemumgebungen angepasst werden
- Sicherheit (Safety)
  - Softwaresystem vermeidet (unter festgelegten Bedingungen) Zustände, die für menschliches Leben, Gesundheit, Eigentum oder Umwelt gefährlich sein können

Quelle: (ISO/IEC 25010:2023)

ISO/IEC 25010:2023 – Product Quality Model

- **Functional suitability**
  - Functional completeness
  - Functional correctness
  - Functional appropriateness
- **Performance efficiency**
  - Time behaviour
  - Resource utilization
  - Capacity
- **Compatibility**
  - Co-existence
  - Interoperability
- **Interaction capability**
  - Appropriateness recognizability
  - Learnability
  - Operability
  - User error protection
  - User engagement
  - Inclusivity
  - User assistance
  - Self-descriptiveness
- **Reliability**
  - Faultlessness
  - Availability
  - Fault tolerance
  - Recoverability
- **Security**
  - Confidentiality
  - Integrity
  - Non-repudiation
  - Accountability
  - Authenticity
  - Resistance
- **Maintainability**
  - Modularity
  - Reusability
  - Analysability
  - Modifiability
  - Testability
- **Flexibility**
  - Adaptability
  - Scalability
  - Installability
  - Replaceability
- **Safety**
  - Operational constraint
  - Risk identification
  - Fail safe
  - Hazard warning
  - Safe integration

Quelle: (ISO/IEC 25010:2023)

Entscheidung für neue Software

- Ziel: minimiere Gesamtkosten bei maximalem Nutzen
- Entscheidung »Make or Buy«
  - Kann man ein Softwaresystem kaufen bzw. lizensieren, das wesentliche Anforderungen erfüllt?
  - Kann man existierende Softwaresysteme entsprechend anpassen oder kombinieren?
  - Sollte man das vollständige Softwaresystem selbst entwickeln bzw. entwickeln lassen (»Individualsoftware«)? Oder Teile davon?
- Commercial-off-the-Shelf (COTS)
  - Mit Konfiguration der spezifischen Kundenbedürfnisse (Customizing)
  - Hierzu zählt auch Software as a Service (SaaS) beim Cloud Computing
- Total Cost of Ownership (TCO)
  - Kalkulation zum Return on Investment (ROI)
  - Direkte & indirekte Kosten für Standard- oder Individualsoftware, insbesondere Startup Costs, Operational Costs und ggf. Retirement Costs

Nutzung existierender Softwarekomponenten

Vorteile

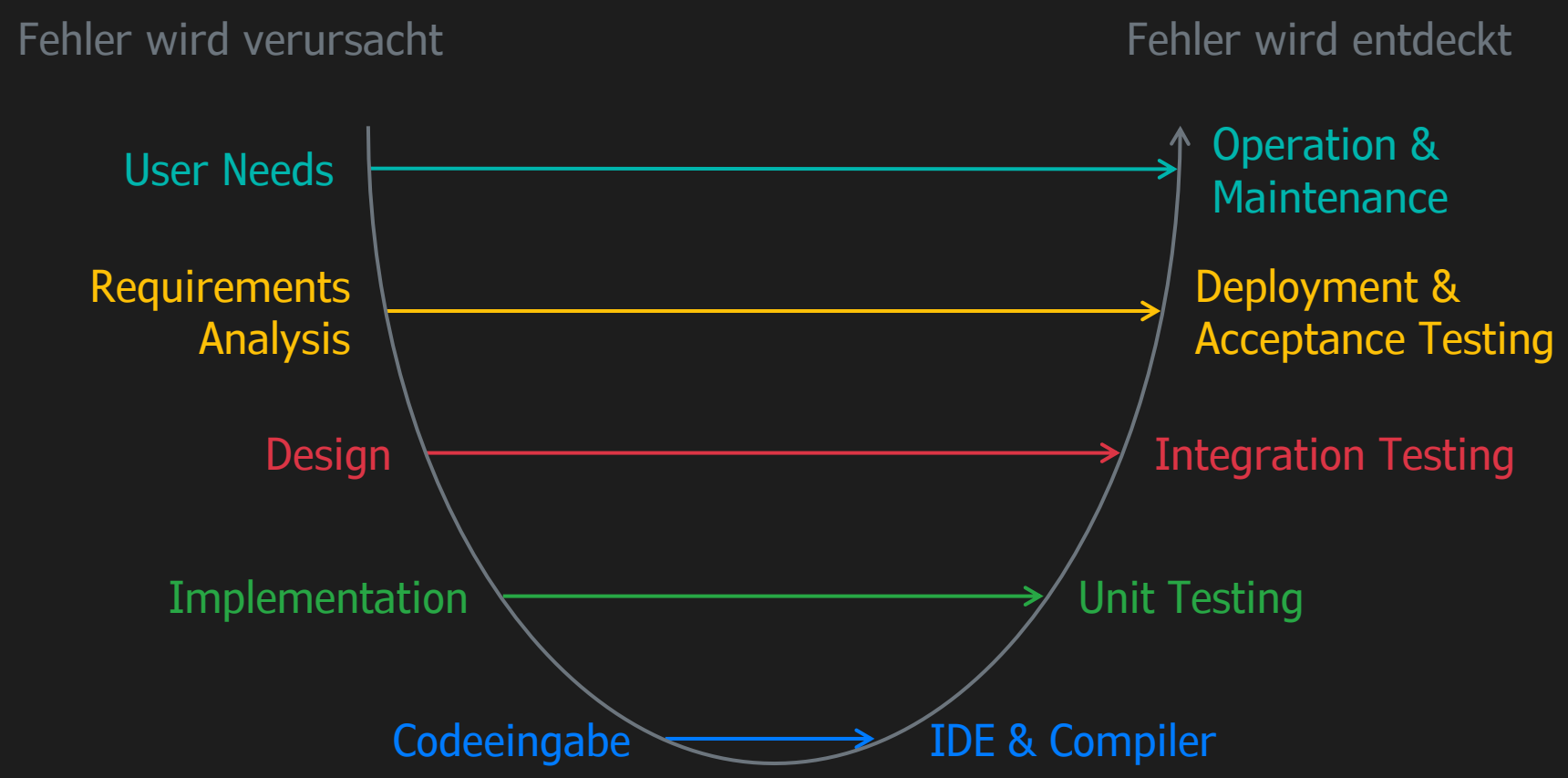
- Schnellerer Einsatz
- Kosten von Individualsoftware wird unterschätzt
- Qualitätsverbesserung durch viele Nutzende
- Entwicklungskosten werden von großer Kundschaft getragen
- Höhere Planungssicherheit
- TCO im Allgemeinen geringer

Nachteile

- Abhängigkeit vom Hersteller: Wartung, Weiterentwicklung, Support
- Individuelle Anpassungen selten möglich
- Weiterentwicklung muss mit vielen Personen abgestimmt werden
- Kein Alleinstellungsmerkmal im Wettbewerb
- Lock-In-Effekt

Softwarefehler: Verursachung und Entdeckung

- Phasen der »Verursachung« und »Entdeckung« von Softwarefehlern



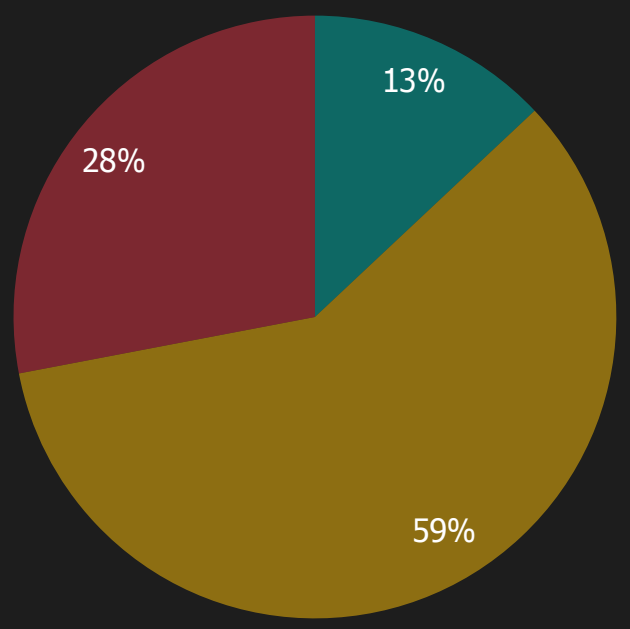
Relative Fehlerbehebungskosten

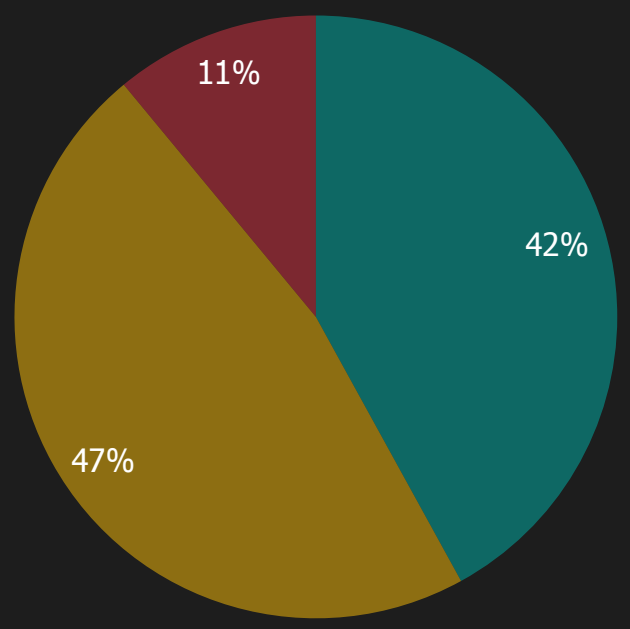
Phase der Fehlerentdeckung	Kostenfaktor
Requirements Analysis	1
Design	3 – 6
Implementation	10
Unit Testing & Integration Testing	15 – 40
Deployment & Acceptance Testing	30 – 70
Operation & Maintenance	40 – 1.000

- Analyse von 63 Softwareprojekten (u.a. IBM)
- Beispiel: eine falsche Annahme, die in Anforderungsanalyse für 100 € behoben werden könnte, kann Fehlerbehebungskosten von 4.000 € bis 100.000 € im Betrieb verursachen
- Siehe auch »Rule of Ten«

Quelle: (Boehm 1984)

CHAOS 2020: IT-Projekte

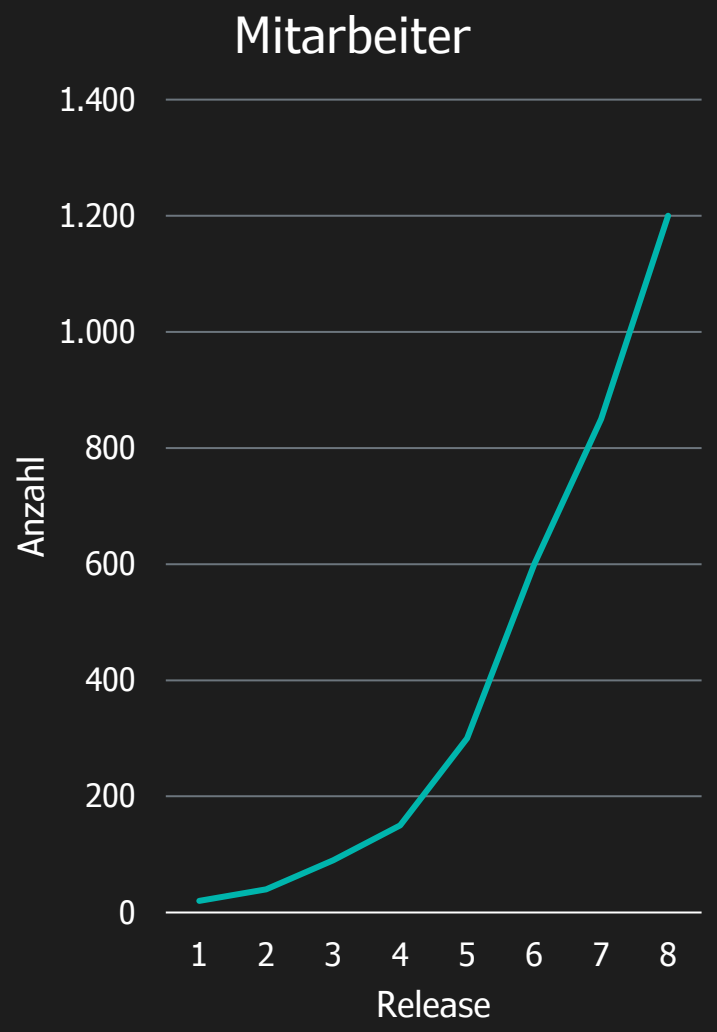


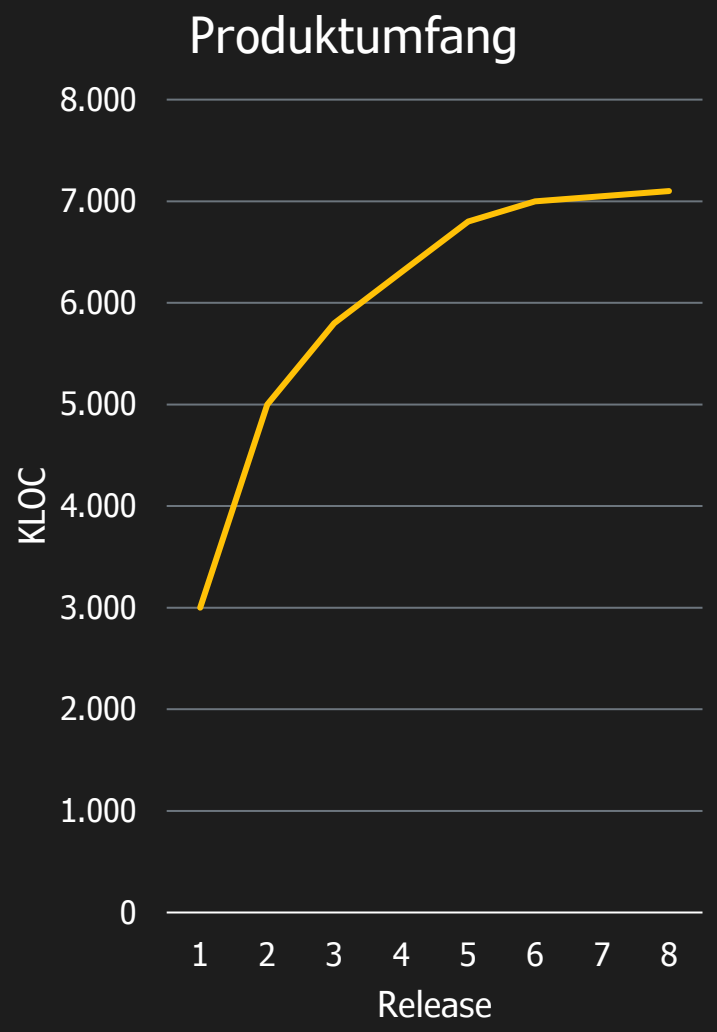


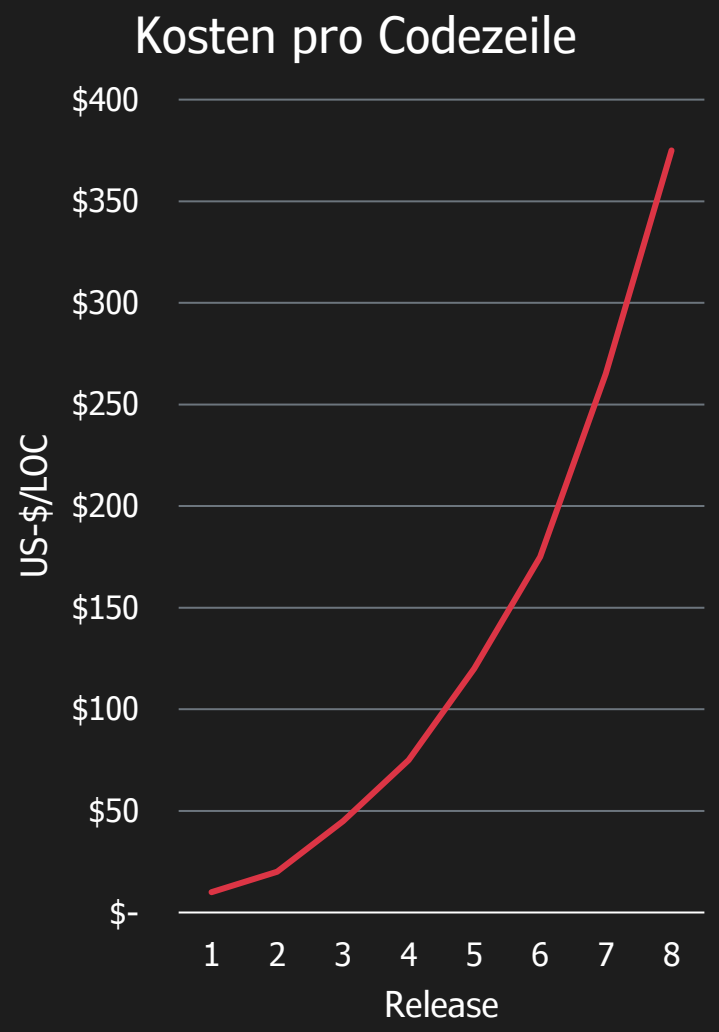
- Erfolgsfaktoren
  - Einbindung der Nutzenden
  - Unterstützung durch das obere Management
  - Klare Anforderungen
- Gründe für das Scheitern
  - Fehlende Zuarbeit durch Stakeholder
  - Unvollständige oder unklare Anforderungen
  - Häufige Anforderungs-änderungen

Quelle: (Johnson 2021)

Fallstudie: marktführendes Softwareprodukt







Quelle: (Martin 2018)

Kosten durch schlechte Softwarequalität

- »Cost of Poor Software Quality in the US«, Consortium for Information & Software Quality, 2022
- Gesamtkosten: 2,41 Billionen US-$ (ca. 10% des BIP der USA)
  - Operational Failures: 1,81 Billionen US-$
  - Legacy Systems: 520 Milliarden US-$
  - Unsuccessful Development Projects: 260 Milliarden US-$
  - Fehlersuche & Bugfixing: 607 Milliarden US-$
- Technische Schulden: deutlich gewachsen auf insgesamt 1,52 Billionen US-$
- Prognose 2025: 40% des gesamten IT-Budgets der USA nur für die »Zinslast der technischen Schulden« (ohne diese Schulden dadurch abzubauen)
- Ca. ein Drittel der Arbeitszeit von Entwickelnden nur für technische Schulden

Quelle: (Krasner 2022)